CS 189 Introduction to Machine Learning
Spring 2024 Jonathan Shewchuk

HW3

Due: Friday, February 23 at 11:59 pm

- Homework 3 consists of coding assignments and math problems.
- We prefer that you typeset your answers using LaTeX or other word processing software. If you haven't yet learned LaTeX, one of the crown jewels of computer science, now is a good time! Neatly handwritten and scanned solutions will also be accepted.
- In all of the questions, **show your work**, not just the final answer.
- The assignment covers concepts on Gaussian distributions and classifiers. Some of the material may not have been covered in lecture; you are responsible for finding resources to understand it.
- **Start early; you can submit models to Kaggle only twice a day!**

Deliverables:

1. Submit your predictions for the test sets to Kaggle as early as possible. Include your Kaggle scores in your write-up. The Kaggle competition for this assignment can be found at
   - MNIST: https://www.kaggle.com/competitions/cs189-hw3-mnist-spring-2024/
   - SPAM: https://www.kaggle.com/competitions/cs189-hw3-spam-spring-2024/
2. Write-up: Submit your solution in **PDF** format to "Homework 3 Write-Up" in Gradescope.
   - On the first page of your write-up, please list students with whom you collaborated
   - Start each question on a new page. If there are graphs, include those graphs on the same pages as the question write-up. DO NOT put them in an appendix. We need each solution to be self-contained on pages of its own.
   - **Only PDF uploads to Gradescope will be accepted**. You are encouraged use LaTeX or Word to typeset your solution. You may also scan a neatly handwritten solution to produce the PDF.
   - **Replicate all your code in an appendix**. Begin code for each coding question in a fresh page. Do not put code from multiple questions in the same page. When you upload this PDF on Gradescope, *make sure* that you assign the relevant pages of your code from appendix to correct questions.
   - While collaboration is encouraged, *everything* in your solution must be your (and only your) creation. Copying the answers or code of another student is strictly forbidden. Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

3. Code: Submit your code as a .zip file to "Homework 3 Code". The code must be in a form that enables the readers to compile (if necessary) and run it to produce your Kaggle submissions.
    - **Set a seed for all pseudo-random numbers generated in your code.** This ensures your results are replicated when readers run your code. For example, you can seed `numpy` with `np.random.seed(42)`.
    - Include a README with your name, student ID, the values of random seed you used, and instructions for compiling (if necessary) and running your code. If the data files need to be anywhere other than the main directory for your code to run, let us know where.
    - Do **not** submit any data files. Supply instructions on how to add data to your code.
    - Code requiring exorbitant memory or execution time might not be considered.
    - Code submitted here must match that in the PDF Write-up. The Kaggle score will not be accepted if the code provided a) does not compile/run or b) runs but does not produce the file submitted to Kaggle.

1 Honor Code

Declare and sign the following statement (Mac Preview, PDF Expert, and FoxIt PDF Reader, among others, have tools to let you sign a PDF file):

"I certify that all solutions are entirely my own and that I have not looked at anyone else's solution. I have given credit to all external sources I consulted."

Signature: _____

2 Gaussian Classification

Let $f_{X|Y=C_i}(x) \sim \mathcal{N}(\mu_i, \sigma^2)$ for a two-class, one-dimensional ($d = 1$) classification problem with classes C_1 and C_2, $P(Y = C_1) = P(Y = C_2) = 1/2$, and $\mu_2 > \mu_1$.

1. Find the Bayes optimal decision boundary and the corresponding Bayes decision rule by finding the point(s) at which the posterior probabilities are equal. Use the 0-1 loss function.

2. Suppose the decision boundary for your classifier is $x = b$. The Bayes error is the probability of misclassification, namely,

$$P_e = P((C_1 \text{ misclassified as } C_2) \cup (C_2 \text{ misclassified as } C_1)).$$

   Show that the Bayes error associated with this decision rule, in terms of b, is

$$P_e(b) = \frac{1}{2\sqrt{2\pi}\sigma}\left(\int_{-\infty}^{b} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma^2}\right) dx + \int_{b}^{\infty} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma^2}\right) dx\right).$$

3. Using the expression above for the Bayes error, calculate the optimal decision boundary b^* that minimizes $P_e(b)$. How does this value compare to that found in part 1? *Hint: $P_e(b)$ is convex for $\mu_1 < b < \mu_2$.*

1. $Q_{C_1}(x) = -\frac{1}{2}\frac{|x-\mu_{C_1}|^2}{\sigma^2} + \Pi_{C_1}$

$Q_{C_2}(x) = -\frac{1}{2}\frac{|x-\mu_{C_2}|^2}{\sigma^2} + \Pi_{C_2}$

$\Pi_{C_1} = P(Y = C_1) = \frac{1}{2}$

$\Pi_{C_2} = P(Y = C_2) = \frac{1}{2}$

Decision boundary: $Q_{C_1}(x) = Q_{C_2}(x)$

$\Rightarrow (x-\mu_{C_1})^2 = (x-\mu_{C_2})^2$

$\Rightarrow x = \frac{\mu_{C_1}+\mu_{C_2}}{2}$

Decision rule: $r^*(x) = \begin{cases} C_1 & Q_{C_1} > Q_{C_2} \text{ 即 } x < \frac{\mu_1+\mu_2}{2} \\ C_2 & x \geq \frac{\mu_1+\mu_2}{2} \end{cases}$

2. misclassify: $x < \frac{\mu_1+\mu_2}{2} = b$, choose C_2

$x > \frac{\mu_1+\mu_2}{2} = b$, choose C_1

$\Rightarrow P = P((C_1/C_2) \cup (C_2/C_1))$

$= P(C_1/C_2) + P(C_2/C_1) = P(\setminus C_2 | Y = C_1) P(Y = C_1)$

$+ P(\setminus C_1 | Y = C_2) P(Y = C_2)$

$= \left(\frac{1}{\sqrt{2\pi}\sigma}\int_{-\infty}^{b} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma^2}\right) dx + \frac{1}{\sqrt{2\pi}\sigma}\int_{b}^{+\infty} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma^2}\right) dx\right) \cdot \frac{1}{2} = \#$

3. optimal b^* requires $P_e(b^*)$ → minimum

$$\frac{d}{db} P_e(b) \sim \frac{d}{db} \int_{-\infty}^{b} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma^2}\right) dx + \frac{d}{db} \int_{b}^{+\infty} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma^2}\right) dx$$

$$\begin{cases} \frac{d}{db} \int_{-\infty}^{b} f(x)dx = f(b) \\ \frac{d}{db} \int_{b}^{+\infty} f(x)dx = -f(b) \end{cases}$$

$$= \exp\left(-\frac{(b-\mu_2)^2}{2\sigma^2}\right) - \exp\left(-\frac{(b-\mu_1)^2}{2\sigma^2}\right) = 0.$$

$$\Rightarrow (b-\mu_2)^2 = (b-\mu_1)^2 \Rightarrow b = \frac{\mu_1+\mu_2}{2} \quad \#.$$

3 Classification and Risk

Suppose we have a classification problem with classes labeled $1, \ldots, c$ and an additional "doubt" category labeled $c+1$. Let $r : \mathbb{R}^d \to \{1, \ldots, c+1\}$ be a decision rule. Define the loss function

$$L(r(x) = i, y = j) = \begin{cases} 0 & \text{if } i = j \quad i, j \in \{1, \ldots, c\}, \\ \lambda_r & \text{if } i = c+1, \\ \lambda_s & \text{otherwise,} \end{cases}$$

where $\lambda_r \geq 0$ is the loss incurred for choosing doubt, $\lambda_s \geq 0$ is the loss incurred for making a misclassification, and at least one of them is nonzero. Hence the risk of classifying a new data point x as class $i \in \{1, 2, \ldots, c+1\}$ is

$$R(r(x) = i|x) = \sum_{j=1}^{c} L(r(x) = i, y = j)\, P(Y = j|x).$$

To be clear, the actual label Y can *never* be $c+1$.

1. Show that the following predictor obtains the minimum risk when $\lambda_r \leq \lambda_s$:
   - Choose class i if $P(Y = i|x) \geq P(Y = j|x)$ for all j and $P(Y = i|x) \geq 1 - \lambda_r/\lambda_s$;
   - Choose doubt otherwise.
2. What happens if $\lambda_r = 0$? What happens if $\lambda_r > \lambda_s$? Explain why this is consistent with what one would expect intuitively.

1. if choose class i

$$R(i|x) = \sum_{j=1}^{c} I(i, y=j)\, P(Y=j|x)$$

$$= \sum_{\substack{j=1 \\ j \neq i}}^{c} \lambda_s\, P(Y=j|x)$$

$$= \lambda_s (1 - P(Y=i|x))$$

if choose doubt

$$R(c+1|x) = \sum_{j=1}^{c} I(c+1, y=j)\, P(Y=j|x)$$

$$= \lambda_r \sum_{j=1}^{c} P(Y=j|x) = \lambda_r$$

choose i means $R(i) \leq R(c+1)$

$\Rightarrow \lambda_s (1 - P(Y=i|x)) \leq \lambda_r$

$\Rightarrow 1 - P(Y=i|x) \leq \frac{\lambda_r}{\lambda_s} \Rightarrow P(Y=i|x) \geq 1 - \frac{\lambda_r}{\lambda_s}$

Obviously we choose $i^* = \arg\max_i P(Y=i|x)$

in order to reduce $R(i|x)$

2. ① $\lambda_r = 0$

now 0-1 loss

$R(i|x) = \lambda_s (1 - P(Y=i|x))$

$=$ misclassify.

$P(Y=i|x) = 1$.

if 100% certain, we choose i, or else choose doubt.

Choosing doubt has no penalty, since $\lambda_r = 0$

② $\lambda_r > \lambda_s$,

then always case 1

which means never choosing doubt.

4 Maximum Likelihood Estimation and Bias

Let $X_1, \ldots, X_n \in \mathbb{R}$ be n sample points drawn independently from univariate normal distributions such that $X_i \sim \mathcal{N}(\mu, \sigma_i^2)$, where $\sigma_i = \sigma / \sqrt{i}$ for some parameter σ. (Every sample point comes from a distribution with a **different variance**.) Note the word "univariate"; we are working in dimension $d = 1$, and each "point" is just a real number.

1. Derive the maximum likelihood estimates, denoted $\hat{\mu}$ and $\hat{\sigma}$, for the mean μ and the parameter σ. (The formulae from class don't apply here, because every point has a different variance.) You may write an expression for $\hat{\sigma}^2$ rather than $\hat{\sigma}$ if you wish—it's probably simpler that way. Show all your work.

2. Given the true value of a statistic θ and an estimator $\hat{\theta}$ of that statistic, we define the *bias* of the estimator to be the the expected difference from the true value. That is,

$$\text{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

   We say that an estimator is *unbiased* if its bias is 0.

   Either prove or disprove the following statement: *The MLE sample estimator $\hat{\mu}$ is unbiased. Hint: Neither the true μ nor true σ^2 are known when estimating sample statistics, thus we need to plug in appropriate estimators.*

3. Either prove or disprove the following statement: *The MLE sample estimator $\hat{\sigma}^2$ is unbiased. Hint: Neither the true μ nor true σ^2 are known when estimating sample statistics, thus we need to plug in appropriate estimators.*

4. Suppose the Variance Fairy drops by to give us the true value of σ^2, so that we only have to estimate μ. Given the loss function $L(\hat{\mu}, \mu) = (\hat{\mu} - \mu)^2$, what is the risk of our MLE estimator $\hat{\mu}$?

1. Draw independently

$$\Rightarrow L(\mu, \sigma; X) = \prod_{i=1}^{n} L(\mu, \sigma_i; X_i)$$

$$= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\frac{\sigma}{\sqrt{i}}} \exp\left(-\frac{(x_i-\mu)^2}{2\sigma^2/i}\right)$$

$$= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}} \frac{\sqrt{i}}{\sigma} \exp\left(-\frac{i(x_i-\mu)^2}{2\sigma^2}\right)$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^n \prod_{i=1}^{n} \sqrt{i} \exp\left(-\frac{\sum_{i=1}^{n}(x_i-\mu)^2 i}{2\sigma^2}\right)$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^n \prod_{i=1}^{n} \sqrt{i} \exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{n} i(x_i-\mu)^2\right)$$

$$\ln L = -n\ln(\sqrt{2\pi}\sigma) + \frac{1}{2}\sum_{i=1}^{n} \ln i - \frac{1}{2\sigma^2}\sum_{i=1}^{n} i(x_i-\mu)^2$$

$$\frac{\partial \ln L}{\partial \mu} = 0 \Rightarrow 2\sum_{i=1}^{n} i(x_i-\mu) = 0$$

$$\Rightarrow \mu\sum_{i=1}^{n} i = \sum_{i=1}^{n} i x_i \Rightarrow \mu = \frac{\sum_{i=1}^{n} i \cdot x_i}{\sum_{i=1}^{n} i}$$

$$\frac{\partial \ln L}{\partial \sigma} = 0 \Rightarrow -\frac{n}{\sigma} + \frac{1}{\sigma^3}\sum_{i=1}^{n} i(x_i-\mu)^2 = 0$$

$$\Rightarrow \sigma^2 = \frac{1}{n}\sum_{i=1}^{n} i(x_i-\mu)^2$$

2. $\text{bias}(\hat{\mu}) = E[\hat{\mu}] - \mu = E\left[\frac{\sum_{i=1}^{n} i \cdot X_i}{\sum_{i=1}^{n} i}\right] - \mu$

$= \frac{1}{\sum_{i=1}^{n} i} E\left[\sum_{i=1}^{n} i \cdot X_i\right] - \mu = \frac{1}{\sum_{i=1}^{n} i} \sum_{i=1}^{n} i \cdot E[X_i] - \mu = E[X_i] - \mu = \mu - \mu = 0$

unbiased.

3. $\text{bias}(\hat{\sigma}^2) = E[\hat{\sigma}^2] - \sigma^2$

$\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} i (X_i - \mu)^2$

we cannot obtain the true μ. therefore $\hat{\mu}$ to institute μ.

$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^{n} i (X_i - \hat{\mu})^2$

$E[\hat{\sigma}^2] = \frac{1}{n} \sum_{i=1}^{n} i \left(E[X_i^2] + E[\hat{\mu}^2] - 2E[X_i \cdot \hat{\mu}] \right) = \frac{1}{n} \sum_{i=1}^{n} i \left(E[X_i^2] + E[\hat{\mu}^2] \right) - \frac{2}{n} E\left[\sum_{i=1}^{n} i X_i \cdot \hat{\mu}\right]$

$\text{Var}[\hat{\mu}] = E[\hat{\mu}^2] - (E[\hat{\mu}])^2$

$= \text{Var}\left[\frac{\sum_{i=1}^{n} i X_i}{\sum_{i=1}^{n} i}\right] = \frac{1}{\left(\sum_{i=1}^{n} i\right)^2} \sum_{i=1}^{n} i^2 \text{Var}[X_i] = \frac{\sum_{i=1}^{n} i^2 \frac{\sigma^2}{i}}{\left(\sum_{i=1}^{n} i\right)^2} = \frac{\sigma^2}{\sum_{i=1}^{n} i}$

$\Rightarrow E[\hat{\mu}^2] = (E[\hat{\mu}])^2 + \frac{1}{\sum_{i=1}^{n} i} = \mu^2 + \frac{\sigma^2}{\sum_{i=1}^{n} i}$

$\frac{\sigma^2}{i} = \text{Var}(X_i) = E[X_i^2] - E[X_i]^2 = E[X_i^2] - \mu^2$

$\Rightarrow E[X_i^2] = \mu^2 + \frac{\sigma^2}{i}$

$E\left[\sum_{i=1}^{n} i X_i \cdot \hat{\mu}\right] = \frac{2}{n(n+1)} E\left[\left(\sum_{i=1}^{n} i X_i\right)\left(\sum_{j=1}^{n} j X_j\right)\right]$

$= \frac{2}{n(n+1)} E\left[\left(\sum_{i=1}^{n} i X_i\right)^2\right]$

$\text{Var}\left[\sum_{i=1}^{n} i X_i\right] = \sum_{i=1}^{n} \text{Var}[i X_i] = \sum_{i=1}^{n} i^2 \text{Var}[X_i] = \sum_{i=1}^{n} i \sigma^2 = \sigma^2 \cdot \frac{n(n+1)}{2}$

$= E\left[\left(\sum_{i=1}^{n} i X_i\right)^2\right] - \left(E\left[\sum_{i=1}^{n} i X_i\right]\right)^2$

$E\left[\sum_{i=1}^{n} i X_i\right] = \sum_{i=1}^{n} i E[X_i] = \mu \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \mu.$

$\Rightarrow E\left[\left(\sum_{i=1}^{n} i X_i\right)^2\right] = \sigma^2 \frac{n(n+1)}{2} + \mu^2 \left[\frac{n(n+1)}{2}\right]^2$

$\Rightarrow E[\hat{\sigma}^2] = \frac{1}{n} \sum_{i=1}^{n} i \cdot \left[\left(\mu^2 + \frac{\sigma^2}{i}\right) + \left(\mu^2 + \frac{\sigma^2}{\sum_{i=1}^{n} i}\right)\right]$

$- \frac{1}{n} \cdot 2 \cdot \frac{2}{n(n+1)} \left[\sigma^2 \frac{n(n+1)}{2} + \mu^2 \left(\frac{n(n+1)}{2}\right)^2\right]$

$$= \frac{1}{n}\left(\cancel{n(n+1)}\mu^2 + n\sigma^2 + \sigma^2\right)$$

$$- \frac{1}{n}\cdot\left(2\sigma^2 + 2\cdot\cancel{\frac{n(n+1)}{2}}\mu^2\right).$$

$$= \frac{n-1}{n}\sigma^2$$

$\Rightarrow$ bias $(\hat{\sigma}^2) = \frac{n-1}{n}\sigma^2 - \sigma^2 = -\frac{1}{n}\sigma^2$ $\leftarrow$ biased.

4. The risk is $E[(\mu-\hat{\mu})^2]$

$E[(\mu-\hat{\mu})^2]$

$= E[\hat{\mu}^2] - 2\mu E[\hat{\mu}] + \mu^2$

$= \mu^2 + \frac{2\sigma^2}{n(n+1)} - 2\mu\cdot\mu + \mu^2 = \frac{2\sigma^2}{n(n+1)}$

The risk is the expected loss $E[L]$.

5 Covariance Matrices and Decompositions

As described in lecture, the covariance matrix $\text{Var}(R) \in \mathbb{R}^{d \times d}$ for a random variable $R \in \mathbb{R}^d$ with mean $\mu \in \mathbb{R}^d$ is

$$\text{Var}(R) = \text{Cov}(R, R) = \mathbb{E}[(R - \mu)(R - \mu)^\top] = \begin{bmatrix} \text{Var}(R_1) & \text{Cov}(R_1, R_2) & \dots & \text{Cov}(R_1, R_d) \\ \text{Cov}(R_2, R_1) & \text{Var}(R_2) & & \text{Cov}(R_2, R_d) \\ \vdots & & \ddots & \vdots \\ \text{Cov}(R_d, R_1) & \text{Cov}(R_d, R_2) & \dots & \text{Var}(R_d) \end{bmatrix},$$

where $\text{Cov}(R_i, R_j) = \mathbb{E}[(R_i - \mu_i)(R_j - \mu_j)]$ and $\text{Var}(R_i) = \text{Cov}(R_i, R_i)$.

If the random variable R is sampled from the multivariate normal distribution $\mathcal{N}(\mu, \Sigma)$ with the PDF

$$f(x) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} e^{-((x-\mu)^\top \Sigma^{-1}(x-\mu))/2},$$

then $\text{Var}(R) = \Sigma$.

Given n points $X_1, X_2, \dots, X_n$ sampled from $\mathcal{N}(\mu, \Sigma)$, we can estimate Σ with the maximum likelihood estimator

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^{n} (X_i - \hat{\mu})(X_i - \hat{\mu})^\top,$$

which is also known as the *sample covariance matrix*.

1. The estimate $\hat{\Sigma}$ makes sense as an approximation of Σ only if $\hat{\Sigma}$ is invertible. Under what circumstances is $\hat{\Sigma}$ not invertible? *Express your answer in terms of the geometric arrangement of the sample points X_i.* We want a geometric characterization, not an algebraic one. Make sure your answer is complete; i.e., it includes all cases in which the covariance matrix of the sample is singular.

2. Suggest a way to fix a singular covariance matrix estimator $\hat{\Sigma}$ by replacing it with a similar but invertible matrix. Your suggestion may be a kludge, but it should not change the covariance matrix too much. Note that infinitesimal numbers do not exist; if your solution uses a very small number, explain how to calculate a number that is sufficiently small for your purposes.

3. Consider the normal distribution $\mathcal{N}(0, \Sigma)$ with mean $\mu = 0$. Consider all vectors of length 1; i.e., any vector x for which $\|x\| = 1$. Which vector(s) x of length 1 maximizes the PDF $f(x)$? Which vector(s) x of length 1 minimizes $f(x)$? Your answers should depend on the properties of Σ. Explain your answer.

4. Suppose we have $X \sim \mathcal{N}(0, \Sigma)$, $X \in \mathbb{R}^n$ and a unit vector $y \in \mathbb{R}^n$. We can compute the projection of the random vector X onto a unit direction vector y as $p = y^\top X$. First, compute the variance of p. Second, with this information, what does the largest eigenvalue $\lambda_{\max}$ of the covariance matrix tell us about the variances of expressions of the form $y^\top X$?

1. $\hat{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(x_i-\hat{\mu})(x_i-\hat{\mu})^T$ is singular

if and only if all the points lie on a common hyperplane in feature space

2. Weighted diagonal correction

$\tilde{\Sigma} = \hat{\Sigma} + \alpha I$

$\hat{\Sigma} \geq 0 \Rightarrow \lambda_i \geq 0$

For invertable matrix $\lambda_i > 0$

For singular matrix $\exists\, \lambda_j = 0$

$\tilde{\Sigma} = \hat{\Sigma} + \alpha I = U\Lambda U^T + \alpha I = U(\Lambda + \alpha I)U^T$

$\tilde{\lambda}_i = \lambda_i + \alpha > 0$ ← become invertable

α should be a small number so not to bias the covariance matrix

choose $\alpha = \varepsilon \cdot \lambda_{max}(\hat{\Sigma})$

hyperparameter tuning: begin with large α, decrease it until $\tilde{\Sigma}^{-1}$ is stable

3. $f(x)$ & $\ln f(x)$ same monotonicity. $\ln f(x) = \square - \frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)$

$\mu = 0$ 时 $\ln f(x) = \square - \frac{1}{2}x^T\Sigma^{-1}x$

$x^T\Sigma^{-1}x$ quadratic form, Σ^{-1} PSD → ellipse

Ellipse's shape is decided by $(\Sigma^{-1})^{-\frac{1}{2}} = \Sigma^{\frac{1}{2}}$

$\vec{x}$ // eigenvector corresponding to $\sqrt{\lambda_{max}}$ → maxmize the PDF

$\vec{x}$ // eigenvector corresponding to $\sqrt{\lambda_{min}}$ → minimize the PDF

4. $E[y^T X] = y^T E[X] = 0$

$Var[p] = E[(p-E[p])^2] = E[(p-E(p))^T(p-E[p])]$

$= E[(X^T - E[X]^T)y\,y^T(X-E[X])]$

$= E[y^T(X-E[X])(X-E[X])^T y]$

$= y^T E[(X-E[X])(X-E[X])^T]y = y^T\Sigma y$

The variances cannot exceed $\lambda_{max}(\Sigma)$ since $y^T\Sigma y \leq y^T\lambda_{max}y = \lambda_{max}y^Ty = \lambda_{max}$

6 Isocontours of Normal Distributions

Let $f(\mu, \Sigma)$ be the probability density function of a normally distributed random variable in $\mathbb{R}^2$. Write code to plot the isocontours of the following functions, each on its own separate figure. Make sure it is clear which figure belongs to which part. You're free to use any plotting libraries or stats utilities you like; for instance, in Python you can use Matplotlib and SciPy. Choose the boundaries of the domain you plot large enough to show the interesting characteristics of the isocontours (use your judgment). Make sure we can tell what isovalue each contour is associated with—you can do this with labels or a colorbar/legend.

1. $f(\mu, \Sigma)$, where $\mu = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$.
2. $f(\mu, \Sigma)$, where $\mu = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$ and $\Sigma = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}$.
3. $f(\mu_1, \Sigma_1) - f(\mu_2, \Sigma_2)$, where $\mu_1 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, $\mu_2 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$ and $\Sigma_1 = \Sigma_2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$.

   (Yes, this is a difference between two PDFs. No, it is not itself a valid PDF. Just plot its isocontours anyway.)
4. $f(\mu_1, \Sigma_1) - f(\mu_2, \Sigma_2)$, where $\mu_1 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, $\mu_2 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$, $\Sigma_1 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ and $\Sigma_2 = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}$.
5. $f(\mu_1, \Sigma_1) - f(\mu_2, \Sigma_2)$, where $\mu_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mu_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$, $\Sigma_1 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ and $\Sigma_2 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

7 Eigenvectors of the Gaussian Covariance Matrix

Consider two one-dimensional random variables $X_1 \sim \mathcal{N}(3, 9)$ and $X_2 \sim \frac{1}{2}X_1 + \mathcal{N}(4, 4)$, where $\mathcal{N}(\mu, \sigma^2)$ is a Gaussian distribution with mean μ and variance σ^2. (This means that you have to draw X_1 first and use it to compute a random X_2.)

Write a program that draws $n = 100$ random two-dimensional sample points from (X_1, X_2). For each sample point, the value of X_2 is a function of the value of X_1 for that *same* sample point, but the sample points are independent of each other. In your code, make sure to choose and set a fixed random number seed for whatever random number generator you use, so your simulation is reproducible, and document your choice of random number seed and random number generator in your write-up. For each of the following parts, include the corresponding output of your program.

1. Compute the mean (in $\mathbb{R}^2$) of the sample.
2. Compute the 2×2 covariance matrix of the sample (based on the sample mean, not the true mean—which you would not know given real-world data).
3. Compute the eigenvectors and eigenvalues of this covariance matrix.
4. On a two-dimensional grid with a horizonal axis for X_1 with range $[-15, 15]$ and a vertical axis for X_2 with range $[-15, 15]$, plot

   (i) all $n = 100$ data points, and

   (ii) arrows representing both covariance eigenvectors. The eigenvector arrows should originate at the mean and have magnitudes equal to their corresponding eigenvalues.

   Hint: make *sure* your plotting software is set so the figure is square (i.e., the horizontal and vertical scales are the same). Not doing that may lead to hours of frustration!
5. Let $U = [v_1 \quad v_2]$ be a 2×2 matrix whose columns are the **unit** eigenvectors of the covariance matrix, where v_1 is the eigenvector with the larger eigenvalue. We use $U^\top$ as a rotation matrix to rotate each sample point from the (X_1, X_2) coordinate system to a coordinate system aligned with the eigenvectors. (As $U^\top = U^{-1}$, the matrix U reverses this rotation, moving back from the eigenvector coordinate system to the original coordinate system). *Center* your sample points by subtracting the mean μ from each point; then rotate each point by $U^\top$, giving $x_{\text{rotated}} = U^\top(x - \mu)$. Plot these rotated points on a new two dimensional-grid, again with both axes having range $[-15, 15]$. (You are not required to plot the eigenvectors, which would be horizontal and vertical.)

In your plots, **clearly label the axes and include a title**. Moreover, **make sure the horizontal and vertical axis have the same scale!** The aspect ratio should be one.

8 Gaussian Classifiers for Digits and Spam

In this problem, you will build classifiers based on Gaussian discriminant analysis. Unlike Homework 1, you are NOT allowed to use any libraries for out-of-the-box classification (e.g. `sklearn`). You may use anything in `numpy` and `scipy`.

The training and test data can be found with this homework. **Do NOT use the training/test data from Homework 1, as they have changed for this homework.** The starter code is similar to HW1's; we provide `check.py` and `save_csv.py` files for you to produce your Kaggle submission files. Submit your predicted class labels for the test data on the Kaggle competition website and be sure to include your Kaggle display name and scores in your writeup. Also be sure to include an appendix of your code at the end of your writeup.

Reminder: please also select relevant code from the appendix on Gradescope for your answer to each question.

1. Taking pixel values as features (no new features yet, please), fit a Gaussian distribution to each digit class using maximum likelihood estimation. This involves computing a mean and a covariance matrix for each digit class, as discussed in Lecture 9 and Section 4.4 of *An Introduction to Statistical Learning*. Attach the relevant code as your answer to this part.

   Hint: You may, and probably should, contrast-normalize the images before using their pixel values. One way to normalize is to divide the pixel values of an image by the l_2-norm of its pixel values.

2. (Written answer + graph) Visualize the covariance matrix for a particular class (digit). Tell us which digit and include your visualization in your write-up. How do the diagonal terms compare with the off-diagonal terms? What do you conclude from this?

3. Classify the digits in the test set on the basis of posterior probabilities with two different approaches.

   (a) (Graphs) Linear discriminant analysis (LDA). Model the class conditional probabilities as Gaussians $\mathcal{N}(\mu_C, \Sigma)$ with different means μ_C (for class C) and the same pooled within-class covariance matrix Σ, which you compute from a weighted average of the 10 covariance matrices from the 10 classes, as described in Lecture 9.

   In your implementation, you might run into issues of determinants overflowing or underflowing, or normal PDF probabilities underflowing. These problems might be solved by learning about `numpy.linalg.slogdet` and/or `scipy.stats.multivariate_normal.logpdf`.

   To implement LDA, you will sometimes need to compute a matrix-vector product of the form $\Sigma^{-1}x$ for some vector x. You should **not** compute the inverse of Σ (nor the determinant of Σ) as it is not guaranteed to be invertible. Instead, you should find a way to solve the implied linear system without computing the inverse.

   Hold out 10,000 randomly chosen training points for a validation set. (You may reuse your Homework 1 solution or an out-of-the-box library for dataset splitting *only*.)

Classify each image in the validation set into one of the 10 classes. Compute the error rate $(1 - \frac{\text{\# points correctly classified}}{\text{\# total points}})$ on the validation set and plot it over the following numbers of randomly chosen training points: 100, 200, 500, 1,000, 2,000, 5,000, 10,000, 30,000, 50,000. (Expect unpredictability in your error rate when few training points are used.)

(b) (Graphs) Quadratic discriminant analysis (QDA). Model the class conditional probabilities as Gaussians $\mathcal{N}(\mu_C, \Sigma_C)$, where Σ_C is the estimated covariance matrix for class C. (If any of these covariance matrices turn out singular, implement the trick you described in Q7(b). You are welcome to use validation to choose the right constant(s) for that trick.) Repeat the same tests and error rate calculations you did for LDA.

(c) (Written answer) Which of LDA and QDA performed better? Why?

(d) (Written answer + graph) Include a plot of validation error versus the number of training points for each digit. Plot all the 10 curves on the same graph as shown in Figure 1. Which digit is easiest to classify? Write down your answer and suggest why you think it's the easiest digit.

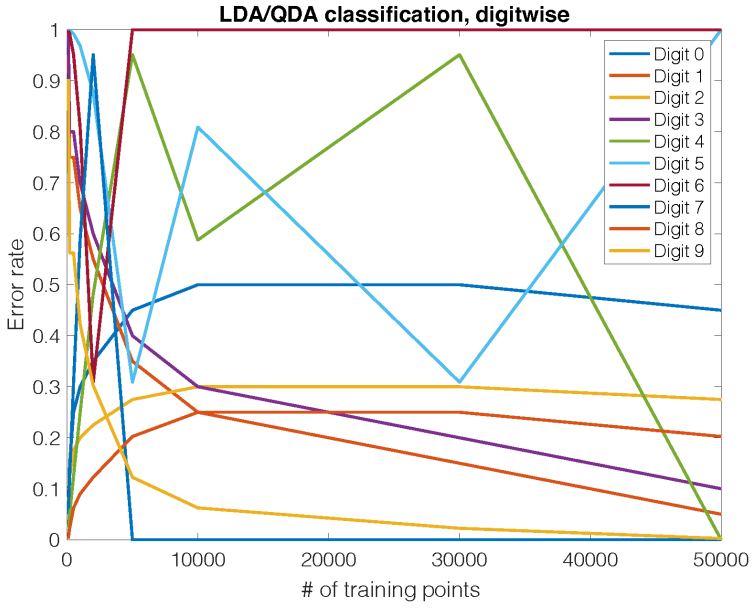

Figure 1: Sample graph with 10 plots

4. (Written answer) With `mnist-data-hw3.npz`, train your best classifier for the `training_data` and classify the images in the `test_data`. Submit your labels to the online Kaggle competition. Record your optimum prediction rate in your write-up and include your Kaggle username. Don't forget to use the "submissions" tab or link on Kaggle to select your best submission!

   You are welcome to compute extra features for the Kaggle competition, as long as they do not use an exterior learned model for their computation (no transfer learning!). If you do so, please describe your implementation in your assignment. Please use extra features **only** for the Kaggle portion of the assignment.

5. (Written answer) Next, apply LDA or QDA (your choice) to spam (`spam-data-hw3.npz`). Submit your test results to the online Kaggle competition. Record your optimum prediction rate in your submission. If you use additional features (or omit features), please describe them. We include a `featurize.py` file (similar to HW1's) that you may modify to create new features.

   Optional: If you use the defaults, expect relatively low classification rates. We suggest using a Bag-Of-Words model. You are encouraged to explore alternative hand-crafted features, and are welcome to use any third-party library to implement them, as long as they do not use a separate model for their computation (no large language models, BERT, or word2vec!).

Submission Checklist

Please ensure you have completed the following before your final submission.

At the beginning of your writeup...

1. Have you copied and hand-signed the honor code specified in Question 1?
2. Have you listed all students (Names and ID numbers) that you collaborated with?

In your writeup for Question 8...

1. Have you included your **Kaggle Score** and **Kaggle Username** for **both** questions 8.4 and 8.5?

At the end of the writeup...

1. Have you provided a code appendix including all code you wrote in solving the homework?
2. Have you included featurize.py in your code appendix if you modified it?

Executable Code Submission

1. Have you created an archive containing all ".py" files that you wrote or modified to generate your homework solutions (including featurize.py if you modified it)?
2. Have you removed all data and extraneous files from the archive?
3. Have you included a README file in your archive briefly describing how to run your code on the test data and reproduce your Kaggle results?

Submissions

1. Have you submitted your test set predictions for both **MNIST** and **SPAM** to the appropriate Kaggle challenges?
2. Have you submitted your written solutions to the Gradescope assignment titled **HW3 Write-Up** and selected pages appropriately?
3. Have you submitted your executable code archive to the Gradescope assignment titled **HW3 Code**?

Congratulations! You have completed Homework 3.

